

AI, 자동화, Rest ful API, 풀스택

Project DreamCollection

01 AI 일정 생성

03 AI 메이트 추천 찾기

02 OCR

04 챗봇

사용자가 조건만을 선택하면, AI가 조건을 기반으로 항공, 숙소, 일정을 전부 선택해주는 시스템

사용자는 일정 선택에 많은 시간을 할당할 필요가 없어집니다.



1. 사용자가 일정 생성 요청

- 기본 조건들을 선택한 후 일정 추천 버튼 선택

2. FastAPI가 요청 스키마 검증

- GenerateRequest가 요청을 검증(프롬프트와 같은 중요한 요소가 빠진다면, ai까지 가지 않고 예외 처리)
- 그렇지 않다면 프롬프트와, 시스템 지시사항, 업무를 Json타입으로 반환

3. Router가 Service 호출

- 프롬프트와 시스템 지시사항을 ai에게 전달

4. Gemini REST API 호출과 응답

- requests.post() 동기식으로 호출하고, 요청 처리를 대기
- 그리고 그 응답에서 사용할 부분만 추출("candidates", "content", "parts", "text" 등을 뽑는다)

5. Pydantic 일정 스키마 검증

- 곧바로 응답하지않고 TripGenerateData로 구조를 확인해 2차 검증

6. Router가 최종 응답 생성

- 검증된 TripGenerateData를 라우터가 GenerateResponse로 감싼다
- FastAPI가 최종 응답을 다시 GenerateResponse 구조에 맞춰 직렬화하고 검증

AI 일정 생성



Ai일정 생성

대한

AI 일정 생성

```
@Override
public TripAiGenerateData recommend(String prompt) {
    if (prompt == null || prompt.isBlank()) {
        return null;
    }

    Map<String, Object> requestBody = Map.of(
        k1: "prompt", prompt,
        k2: "task", v2: "trip_recommend");

    String url = aiServerBaseUrl + "/api/v1/generate";

    try {
        String response = restClient.post()
            .uri(url)
            .header("Content-Type", "application/json")
            .body(requestBody)
            .retrieve()
            .body(String.class);

        TripAiGenerateResponse parsed =
            objectMapper.readValue(response, TripAiGenerateResponse.class);

        if (parsed == null || parsed.data() == null
            || parsed.data().days() == null
            || parsed.data().days().isEmpty()) {
            log.warn("AI server empty data: {}", response);
            return null;
        }

        log.info("Trip AI response day count={}, parsed.data().days().size()", parsed.data().days().size());
        return parsed.data();
    } catch (Exception e) {
        log.error("AI server call failed url={}, url, e);
        return null;
    }
}
```

응답에 프롬프트와 업무를 담아 ai에게 전달합니다

현재 ai-server를 fastApi로 따로 만들었기에 ai-server에 이 주소를 담습니다

post로 요청을 보냅니다

objectMapper로 타입을 맞추어 응답을 받을 수 있도록 해줍니다

```
public List<TripRecommendDTO> build(
    PlanRequestDTO request,
    TripAiGenerateData aiData,
    TripPlacePools pools
) {
    if (pools == null || pools.allPlaces().isEmpty()) {
        log.warn("Place pools are empty. Trip recommendation failed.");
        return List.of();
    }

    if (pools.mealEmpty()) {
        log.warn("Meal place pool is empty. Cannot enforce Lunch/Dinner.");
        return List.of();
    }

    int expectedDayCount = dayCount(request.getWhen());
    List<DayPlanDTO> dayPlans = mapAiDays(aiData, pools, expectedDayCount);

    if (dayPlans.isEmpty() || !isValidSchedule(dayPlans, expectedDayCount)) {
        log.warn("AI schedule validation failed or place matching failed.");
        return List.of();
    }

    String region = request.getDestination() != null && !request.getDestination().isBlank()
        ? request.getDestination()
        : request.getRegion();

    return List.of(TripRecommendDTO.builder()
        .recommendation(1)
        .title((region != null ? region : "Trip") + " recommended itinerary")
        .summary(nullToEmpty(request.getWhen()) + " "
            + nullToEmpty(request.getTheme()) + " recommended itinerary")
        .days(dayPlans)
        .build());
}
```

응답의 반복과 토큰의 사용량을 줄이기위해서 필요한 값이 없다면 그 값들을 우선적으로 요청하고 ai에게 요청합니다

아래 if 문들은 전부 값을 검사하는 부분입니다

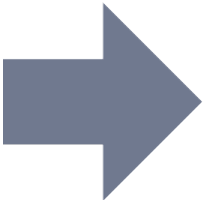
```
@router.post("/generate", response_model=GenerateResponse)
def generate(req: GenerateRequest):

    try:
        system = req.system or DEFAULT_TRIP_SYSTEM
        data = trip_generate_service.generate(
            prompt=req.prompt,
            system=system
        )

        return GenerateResponse(
            task=req.task,
            data=data,
        )

    except Exception as e:
        raise HTTPException(status_code=502, detail=str(e))
```

받아온 프롬프트 값과 선택한 조건들을 전부 정리해 generate()로 생성합니다

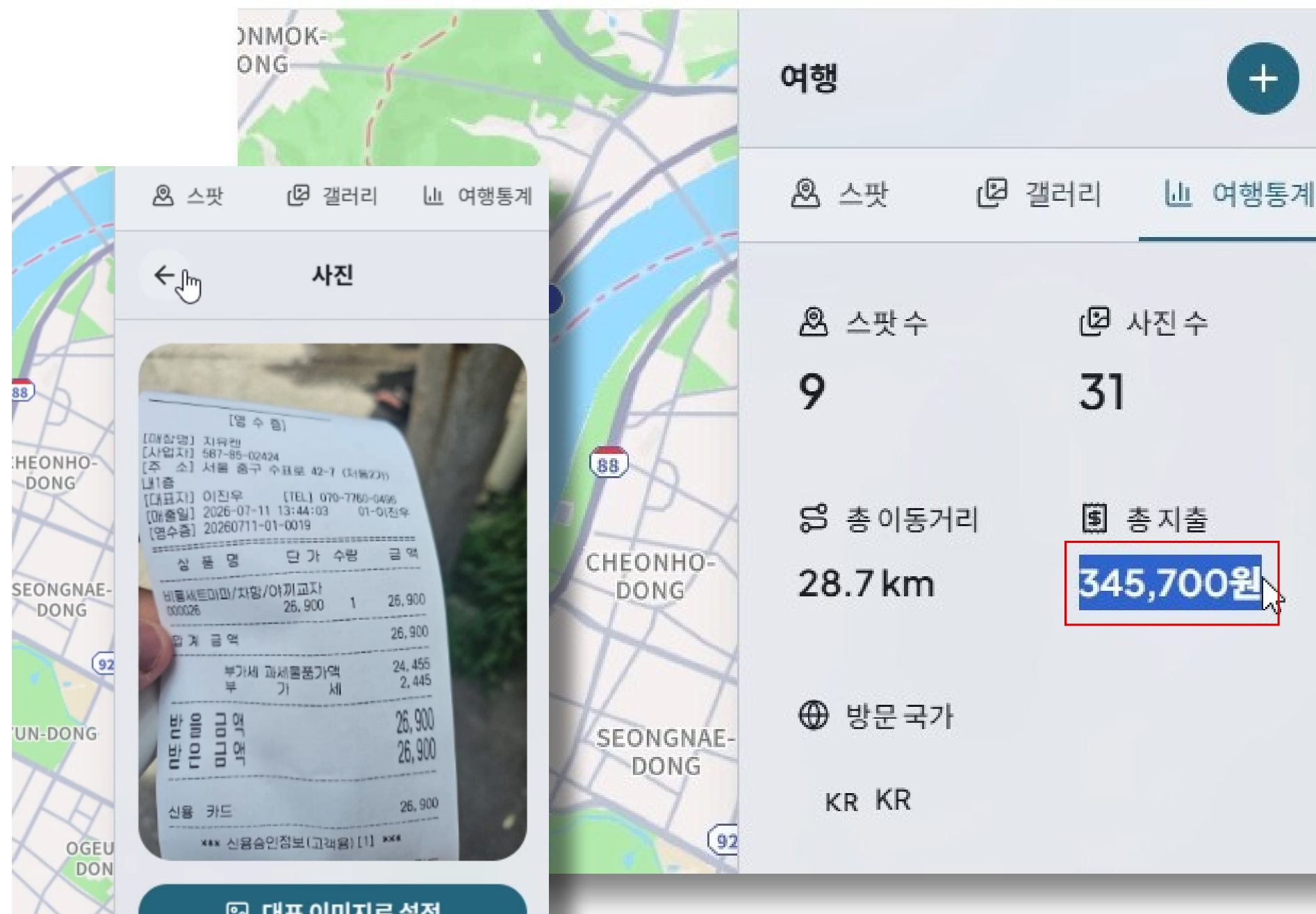


앞선에서 받은 요청을 아래와 같은 Json 타입으로 반환을 합니다 그 이후에 ai에게 전달합니다

```
{
  "prompt": "여행 일정 생성에 필요한 프롬프트",
  "system": "선택적인 시스템 지시사항",
  "task": "trip-generate"
}
```


여행 중 발생한 영수증을 별도의 정리 없이 사진으로 업로드 하면 AI가 정산을 자동으로 해주는 시스템

여행 후 정산에 고역과 시간 할당을 해소 해줍니다



1. 영수증 사진 업로드

- 여행 중 촬영한 영수증 사진 업로드
- OCR 분석 요청

2. OCR 텍스트 추출

- 영수증 이미지에서 문자 추출
- AI 분석을 위한 데이터 생성

3. 영수증 여부 판별

- OCR 결과를 OpenAI에 전달

4. AI 영수증 분석

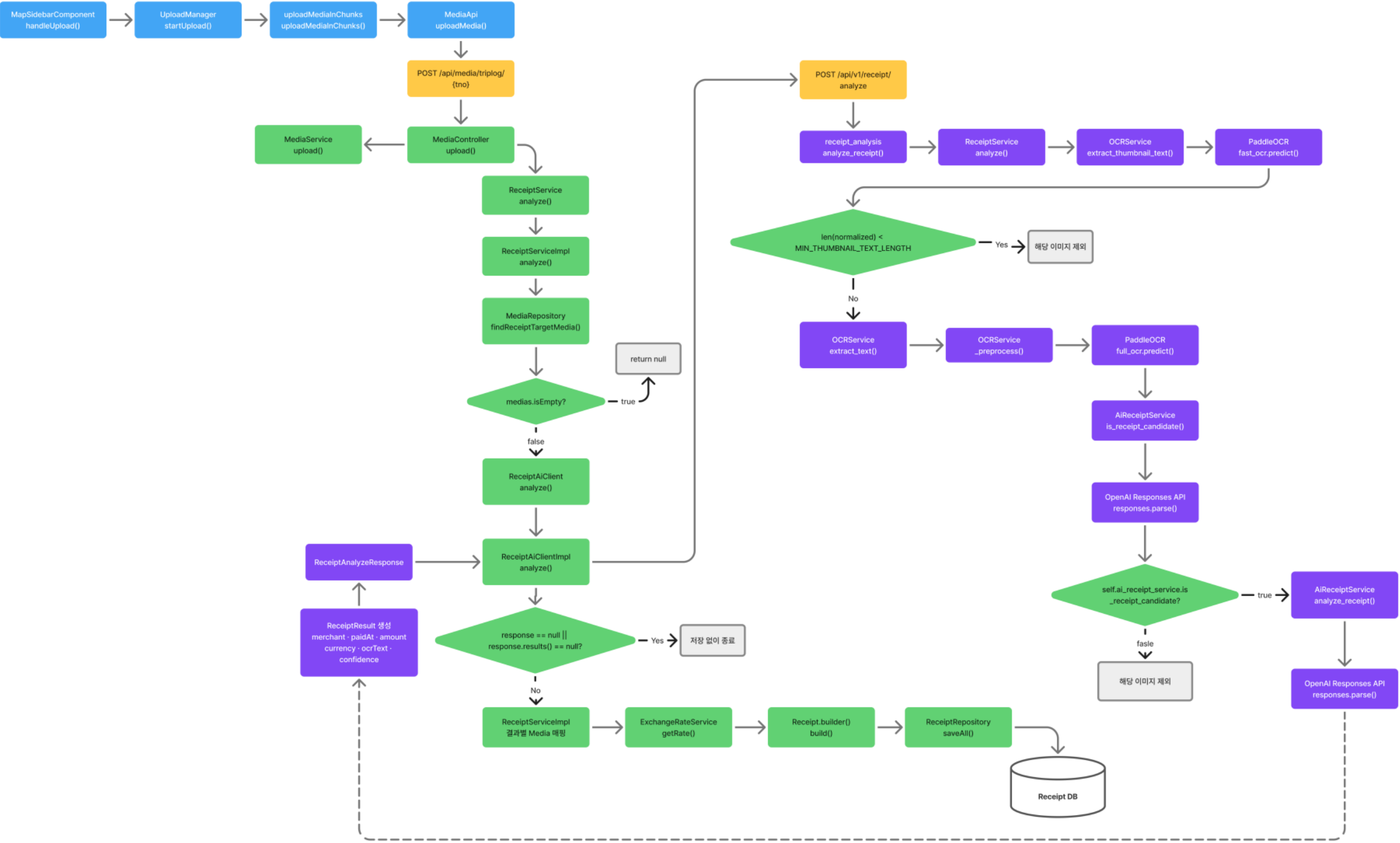
- 영수증 이미지와 OCR 결과를 함께 분석
- 상호명, 금액, 날짜 등 정보 추출

5. 정산 정보 생성

- 분석 결과를 JSON 형태로 반환

6. 사용자에게 결과 제공

- 자동 정산 결과 제공



```
def is_receipt_candidate(self, text: str) -> bool:
    response = self.client.responses.parse(
        model=self.model,
        input=[
            {
                "role": "system",
                "content": [
                    {
                        "type": "input_text",
                        "text": self.filter_prompt,
                    }
                ],
            },
            {
                "role": "user",
                "content": [
                    {
                        "type": "input_text",
                        "text": f"OCR 결과\n\n{text}",
                    }
                ],
            },
        ],
        text_format=ReceiptCandidate,
    )
    return response.output_parsed.candidate
```

```
def analyze_receipt(self, text: str, media_path: str, stored_file_name: str) -> AIReceiptResult:
    image_path = self._resolve_image_path(media_path, stored_file_name)
    image_data = self._encode_image(image_path)
    media_type = self._get_media_type(image_path)

    response = self.client.responses.parse(
        model=self.model,
        input=[
            {
                "role": "system",
                "content": [
                    {
                        "type": "input_text",
                        "text": self.analyze_prompt,
                    }
                ],
            },
            {
                "role": "user",
                "content": [
                    {
                        "type": "input_image",
                        "image_url": f"data:{media_type};base64,{image_data}",
                    },
                    {
                        "type": "input_text",
                        "text": f"OCR 결과\n\n{text}",
                    }
                ],
            },
        ],
        text_format=AIReceiptResult,
    )
    return response.output_parsed
```

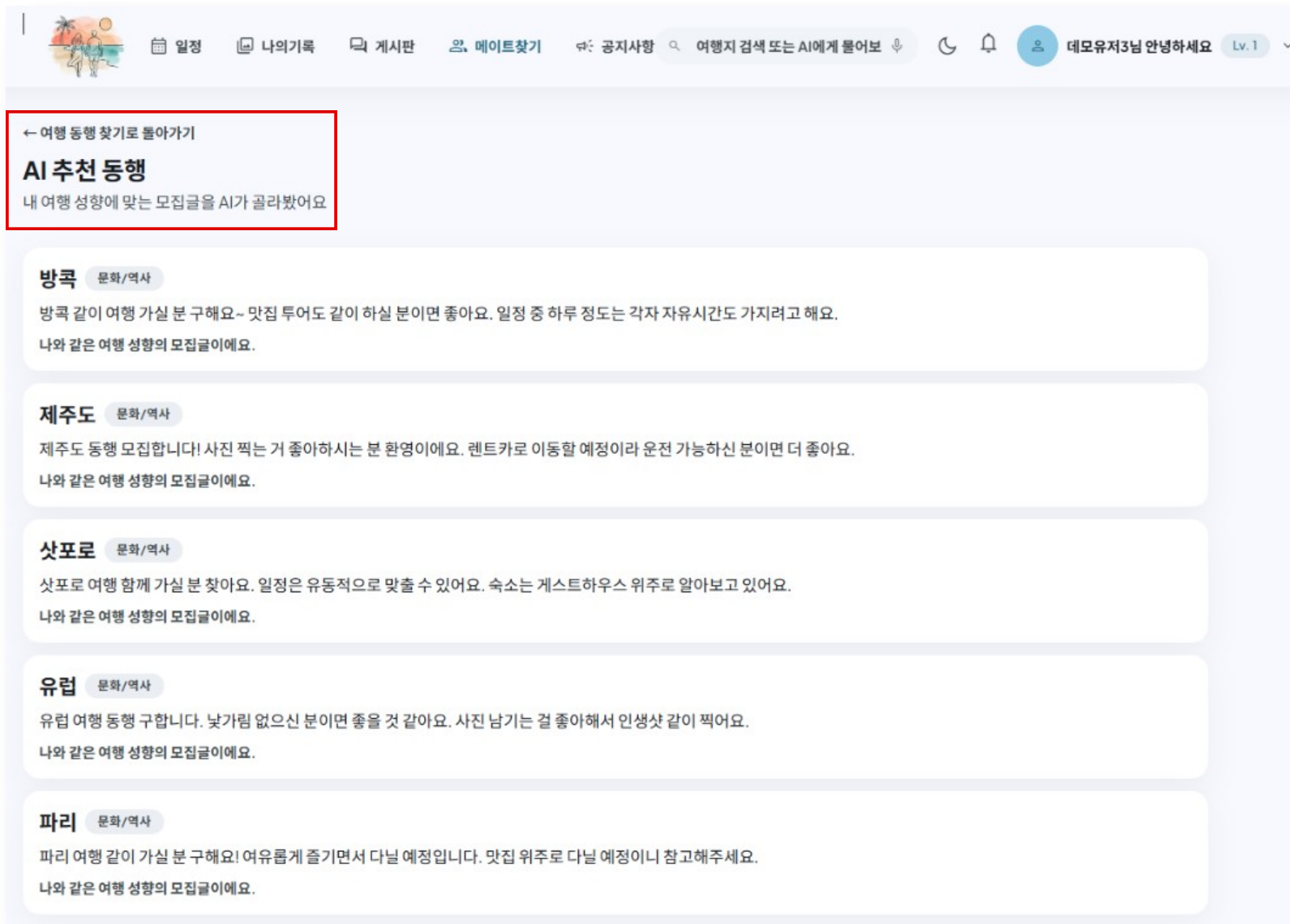
영수증 이미지와 OCR 결과를 함께 OpenAI에 전달하여 상호명, 금액, 날짜 등의 정보를 AI가 자동으로 분석하고 구조화된 데이터로 반환하도록 구현하였습니다

전체적인 구조는 아래와 같습니다



AI가 사용자의 성향과 여행 스타일을 분석해 유사한 성향의 사용자들을 찾아줘 여행 메이트를 찾을수 있게 도와주는 시스템

비슷한 취향의 메이트를 찾기 쉽게 도와줍니다.



1. 사용자가 AI 메이트 추천 요청

- 메이트 추천 버튼 선택
- Spring Boot 서버에서 사용자 성향과 추천 후보 게시글 조회

2. 추천 후보 게시글 선별

- 자신의 게시글을 제외한 최대 30개의 후보를 선별하여 AI 연산 최적화

3. AI 서버 추천 요청

- 사용자 성향과 후보 게시글 정보를 프롬프트로 구성
- FastAPI 기반 AI 서버(/v1/generate)에 추천 요청을 전달하고 Pydantic을 통해 요청 데이터의 스키마를 검증

4. Gemini AI 추천 결과 생성

- Gemini API를 호출하여 사용자 성향과 여행 정보를 기반으로 추천 결과 생성
- 추천 게시글과 추천 이유를 JSON 형식으로 생성

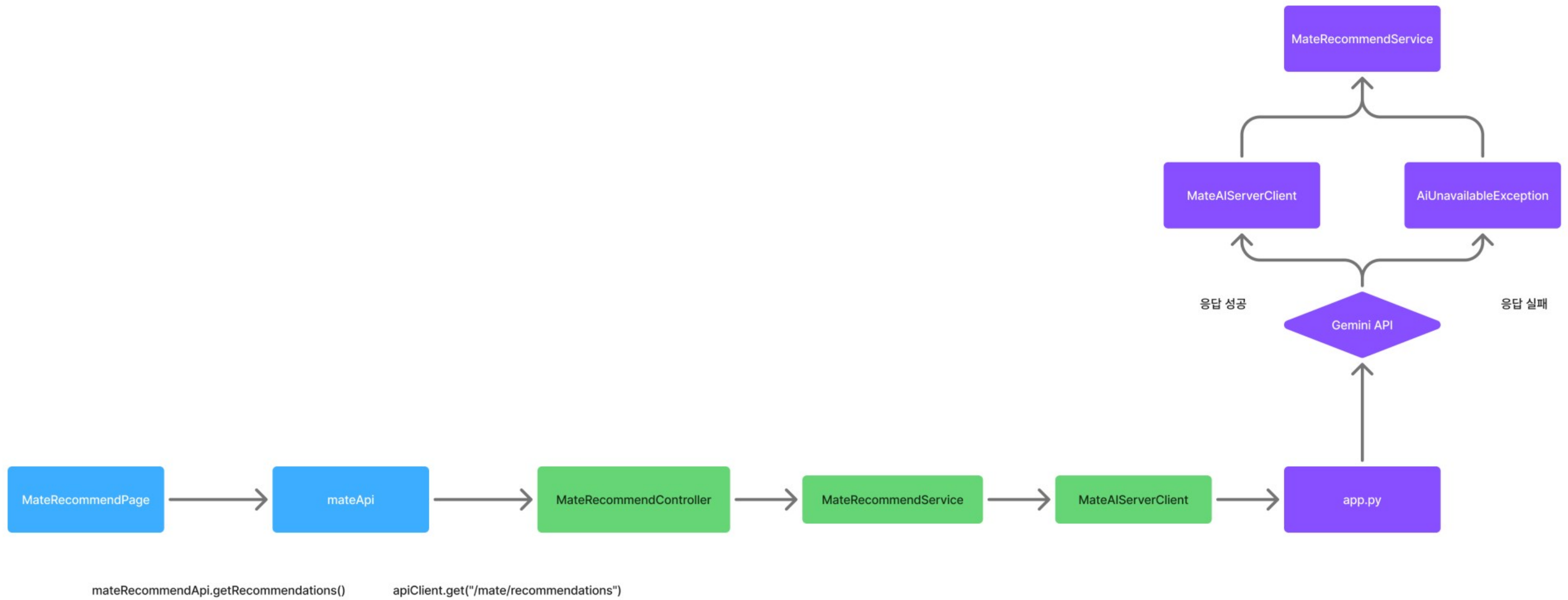
5. FastAPI 응답 반환

- 추천 결과(text)와 작업 정보(task)를 JSON 형태로 반환

6. 최종 추천 결과 제공

- AI 서버 장애 또는 응답 실패 시 폴백(Fallback) 추천을 제공하여 서비스의 연속성을 보장

AI 메이트 추천 찾기



AI 메이트 추천 찾기

```
try {
    aiText = mateAiServerClient.generate(prompt, system, task: "mate_recommend");
} catch (MateAiServerClient.AiUnavailableException e) {
    String status = e.isRateLimited() ? "AI_RATE_LIMITED" : "AI_ERROR";
    log.warn("AI 추천 실패(status={}), 폴백으로 전환", status);
    return MateRecommendResponseDTO.of(fallbackRecommend(myStyle, candidates), status);
}
```

사용자 성향과 추천 후보 게시글 정보를 FastAPI 기반 AI 서버로 전달하고
AI 서버 장애 또는 호출 실패 시 폴백(Fallback) 추천을 제공하여 서비스가 중단되지 않도록 구현했습니다.

```
class GenerateRequest(BaseModel):
    prompt: str
    system: str | None = None
    task: str | None = None

@app.get("/health")
def health():
    return {"ok": True}

@app.post("/v1/generate")
def generate(body: GenerateRequest):
```

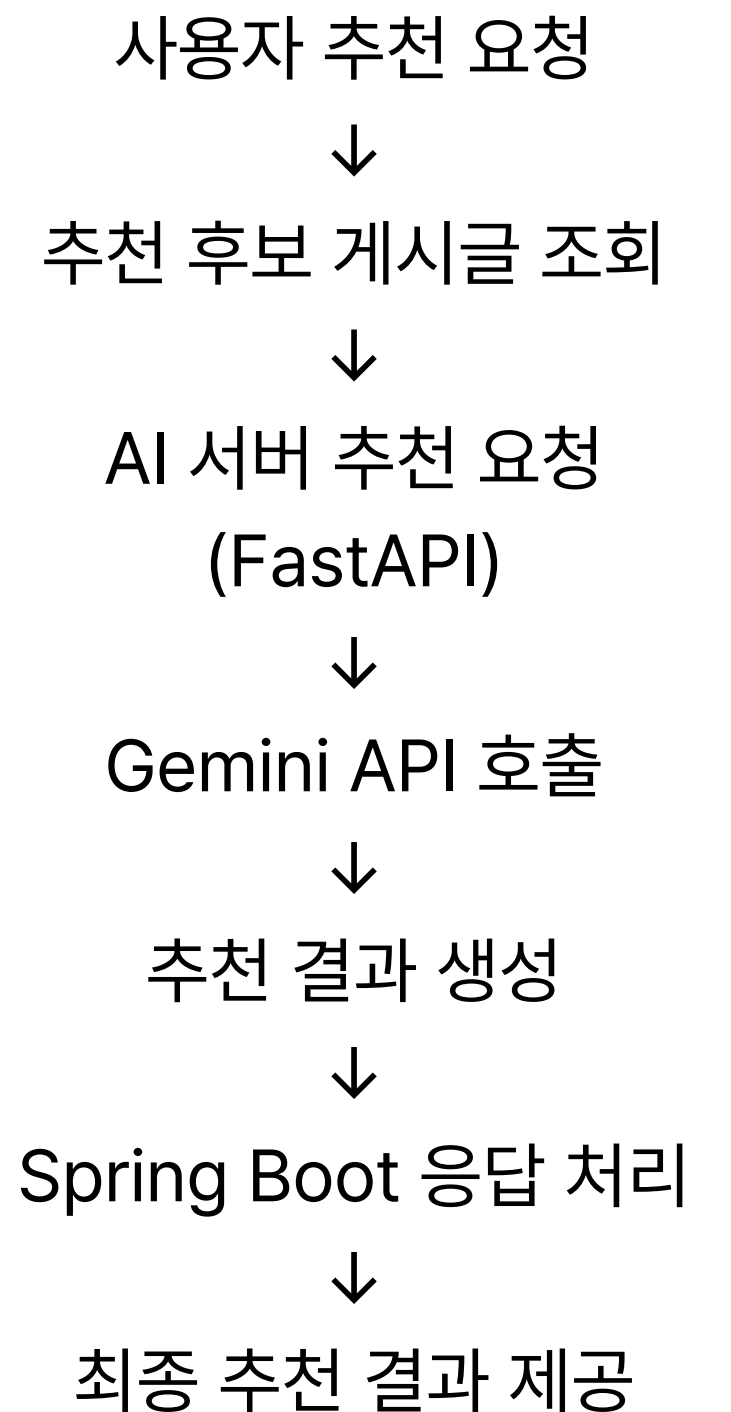
Pydantic을 통해 요청 데이터의 형식을 검증하고
FastAPI 엔드포인트에서 AI 추천 요청을 처리하도록 구현했습니다.

```
try:
    model = genai.GenerativeModel(
        model_name=MODEL_NAME,
        system_instruction=body.system if body.system else None,
    )
    response = model.generate_content(body.prompt)
    text = response.text
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

return {"text": text, "task": body.task}
```

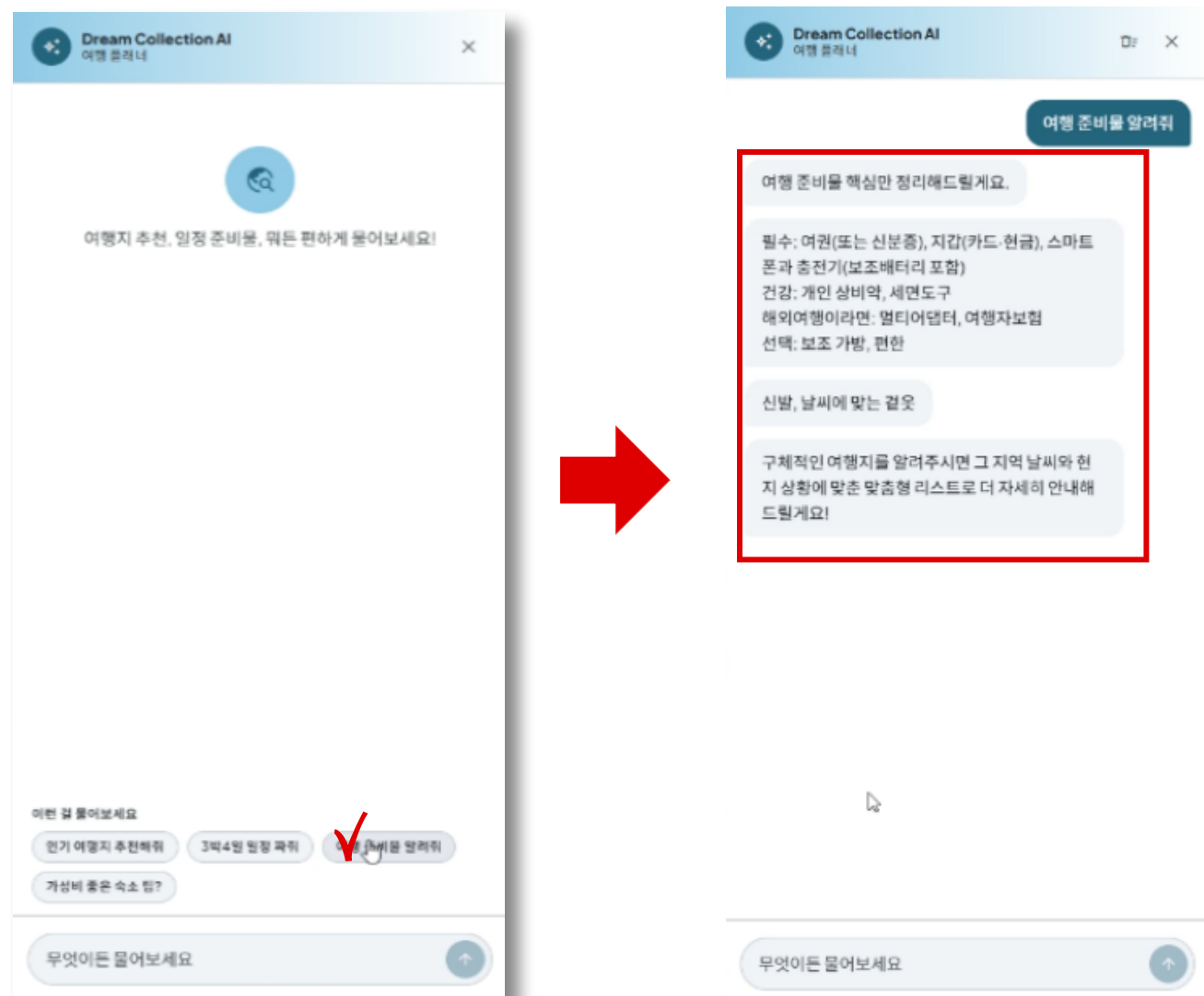
Gemini API를 호출하여 사용자 성향을 기반으로 추천 결과를 생성하고,
생성된 결과를 JSON 형태로 Spring Boot 서버에 반환하도록 구현했습니다

전체적인 구조는 아래와 같습니다

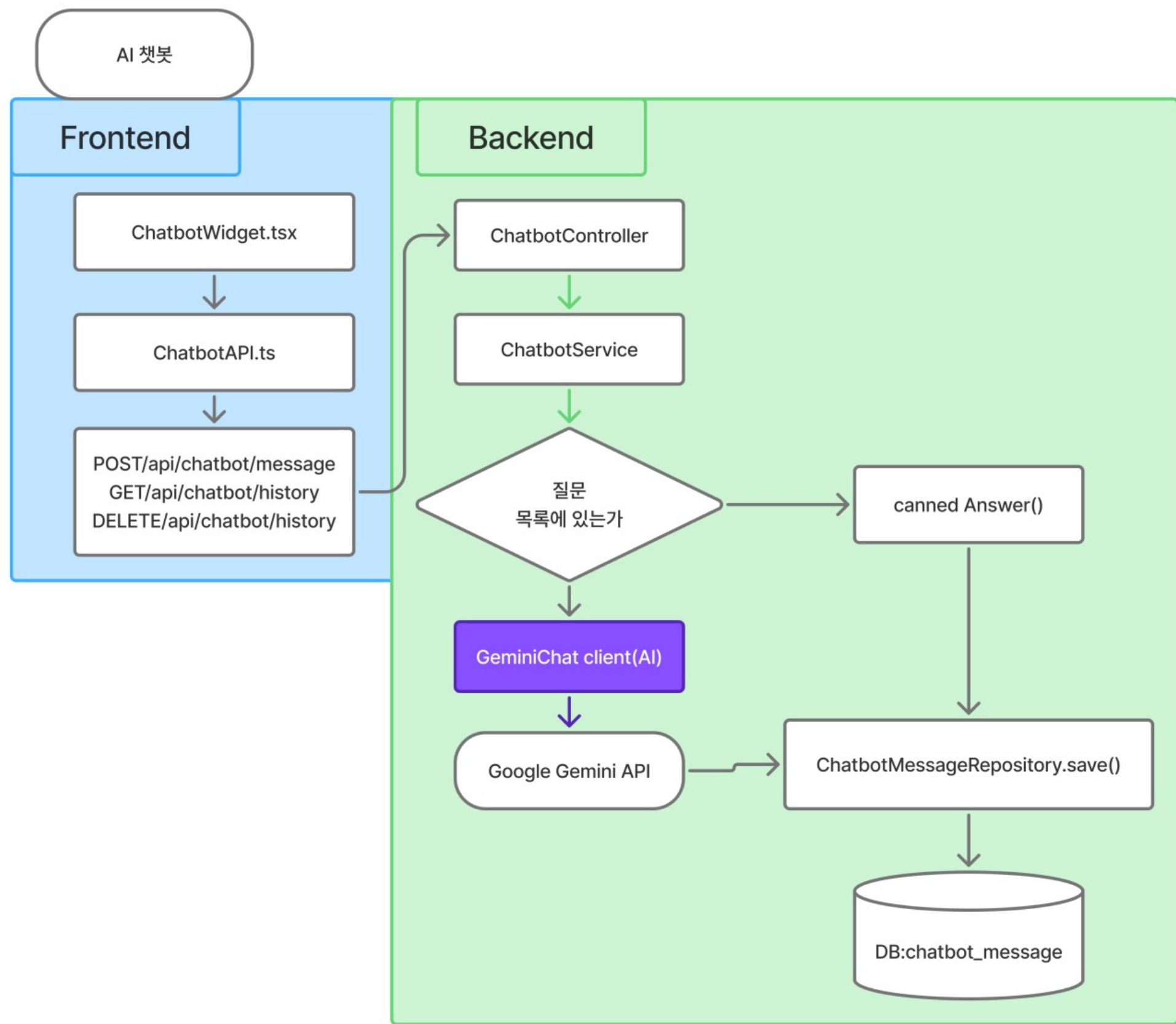


웹사이트 이용 도중 사용에 어려움을 겪고 있거나, 여행지에 관해서 정보를 얻을 수 있는 챗봇 시스템

다양한 상황에서의 사용자의 불편함을 해소해줍니다.



1. 사용자가 질문 입력
 - 여행 정보 또는 사이트 이용 관련 질문 입력
2. 시스템 프롬프트 구성
 - 여행 전문 AI 챗봇 역할 및 응답 규칙 설정
 - 사이트 최신 정보(Context) 추가
3. Gemini AI 요청
 - 대화 내용을 Gemini 형식으로 변환
 - 사용자 질문과 함께 Gemini API 호출
4. AI 답변 생성
 - 질문과 대화 문맥을 분석
 - 맞춤형 여행 정보 및 답변 생성
5. 응답 처리
 - AI 응답(Text) 추출
 - 오류 발생 시 안내 메시지 반환
6. 사용자에게 결과 제공
 - 생성된 답변을 채팅 화면에 출력
 - 대화 이력을 유지하여 연속적인 상담 지원




```
public String chat(List<ChatMessageDto> history, String siteContext) {
    if (apiKey == null // apiKey.isBlank()) {
        return "챗봇 기능을 사용하려면 관리자가 Gemini API 키를 설정해야 해요. "
            + "(백엔드 환경변수 GEMINI_API_KEY)";
    }

    String systemPrompt = (siteContext == null // siteContext.isBlank())
        ? SYSTEM_PROMPT_BASE
        : SYSTEM_PROMPT_BASE + "\n\n" + siteContext;

    List<Map<String, Object>> contents = history.stream()
        .map(m -> Map.of(
            "role", m.role().equals("assistant") ? "model" : "user",
            "parts", List.of(Map.of("text", m.content()))
        ))
        .toList();

    Map<String, Object> requestBody = Map.of(
        "systemInstruction", Map.of("parts", List.of(Map.of("text", systemPrompt))),
        "contents", contents,
        "generationConfig", Map.of("maxOutputTokens", MAX_OUTPUT_TOKENS)
    );

    String url = API_URL_TEMPLATE.formatted(model);

    try {
        String response = restClient.post()
            .uri(url)
            .header("x-goog-api-key", apiKey)
            .header("Content-Type", "application/json")
            .body(requestBody)
            .retrieve()
            .body(String.class);
    }
```

이전 대화 내용과 시스템 프롬프트를 Gemini API 요청 형식으로 구성한 뒤
Google Gemini API를 호출하여 문맥을 고려한 AI 답변 생성을 요청하도록 구현하였습니다.

```
JsonNode node = objectMapper.readTree(response);

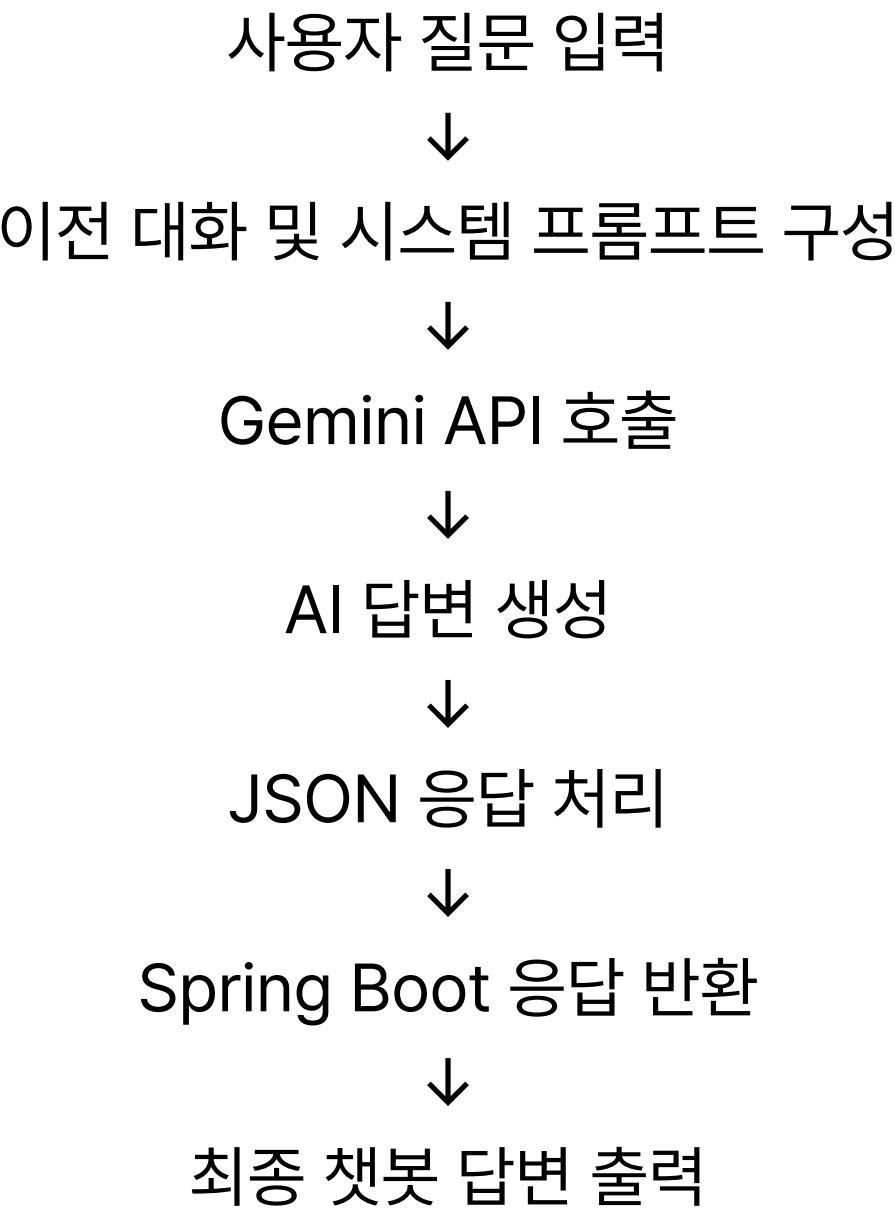
JsonNode text = node.path("candidates").path(0)
    .path("content").path("parts").path(0).path("text");

if (!text.isMissingNode() && !text.asText("").isBlank()) {
    return text.asText();
}

log.warn("Gemini API 응답에서 답변 텍스트를 찾지 못함: {}", response);
return "죄송해요, 답변을 만들지 못했어요. 다시 시도해주세요.";
} catch (Exception e) {
    log.error("Gemini API 호출 실패", e);
    return "일시적으로 챗봇에 연결할 수 없어요. 잠시 후 다시 시도해주세요.";
}
```

Gemini API가 반환한 JSON 데이터에서 AI 답변을 추출하여 사용자에게 전달하고
응답 생성 실패나 API 호출 오류 발생 시 안내 메시지를 반환하도록 구현하였습니다.

전체적인 구조는 아래와 같습니다



Thank You